

Lab Report

Operating Systems Laboratory

Assignment 3: Process Scheduling

Anshuman Rahul (CS24BT060)
Sidharth B (CS24BT061)

1 Introduction

The nature of the scheduling policy a system utilizes has a significant impact on the performance of that system. In this assignment, we implement a discrete event simulator to implement the following policies and test them over 5 workloads

- First In First Out
- Round Robin
- MLFQ

2 Policies

- **First In First Out:** Processes are scheduled in the order they arrive, FIFO is an example of a non-preemptive scheduling policy.
- **Round Robin:** Processes run for a fixed quantum until either the quantum expires or the process terminates. We use a quantum of 2. Round Robin is an example of preemptive policy.
- **MLFQ:** Multiple queues are utilized to introduce a notation of priority, rules are set such that priority of a process changes over time depending on the behavior of the process up to that point. We use 3 queues and each level has a quantum of 2.
 - **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs
 - **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in round-robin fashion using the time slice (quantum length) of the given queue.
 - **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
 - **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
 - **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue).

3 Modelling a Process and Event

3.1 Process

The `Process` structure represents an individual process in the simulation. It stores the process's static attributes, such as its ID, arrival time, and CPU/IO burst sequences, along with its dynamic state, which is updated as the process moves through the scheduler.

Listing 1: Data structure representing an individual process.

```
struct Process {
    int pid;
    int arrival_time;
    std::vector<int> cpu_bursts;
    std::vector<int> io_bursts;

    int io_burst_index = -1;
    int cpu_burst_index = -1;
    int cpu_burst_remaining;

    int epoch = 0;
    int quantum_remaining = 0;
    int cpu_start_time = -1;

    int current_queue = 0;

    int first_service_time = -1;
    int completion_time = -1;
};
```

The fields can be grouped as follows:

- **Static attributes:** `pid`, `arrival_time`, `cpu_bursts`, and `io_bursts` describe the process's identity and its burst pattern.
- **Execution state:** `cpu_burst_index`, `io_burst_index`, and `cpu_burst_remaining` track which burst the process is currently executing and how much of it is left.
- **Scheduling state:** `epoch`, `quantum_remaining`, `cpu_start_time`, and `current_queue` record information used by the scheduling algorithm (e.g., which MLFQ queue the process currently occupies).
- **Metrics:** `first_service_time` and `completion_time` are used to compute performance statistics such as response time and turnaround time.

3.2 Event Types

The `EventType` enumeration defines the kinds of events that can occur during the simulation. Each event in the event queue carries one of these values, and the simulator dispatches on it to decide what action to take when the event is processed. The explicit

numeric values also define a tie-breaking priority: when two events occur at the same simulated time, the one with the lower value is processed first.

Listing 2: Enumeration of simulation event types.

```
enum class EventType {  
    ARRIVAL = 0,  
    IO_COMPLETION = 1,  
    CPU_RELEASE = 2,  
    PRIORITY_BOOST = 3  
};
```

- **ARRIVAL** — a new process enters the system and must be placed on the ready queue.
- **IO_COMPLETION** — a process has finished its IO burst and returns to the ready queue.
- **CPU_RELEASE** — the cpu is done with the process running at that moment.
- **PRIORITY_BOOST** — a periodic aging event that moves all processes to the highest-priority queue to prevent starvation.

3.3 Event

The **Event** structure represents a simulation event. Each event captures something that happens to a process at a specific point in simulated time. Events are maintained in a priority queue ordered by time, so the simulator always processes the earliest event next.

Listing 3: Data structure representing a simulation event.

```
struct Event {  
    int pid;  
    int time;  
    int epoch;  
    int cpu_id;  
    EventType event;  
};
```

- **pid** — the ID of the process this event belongs to.
- **time** — the simulated time at which the event occurs.
- **epoch** — the scheduling epoch of the process, used to distinguish stale events for a process that has since been rescheduled (an event from an older epoch can be safely ignored).
- **cpu_id** — the CPU on which the process was running, so the simulator knows which CPU becomes free.
- **event** — the type of event, one of **ARRIVAL**, **IO_COMPLETION**, **CPU_RELEASE**, or **PRIORITY_BOOST** (see Listing 2).

4 Metrics and Graphs

4.1 Single CPU

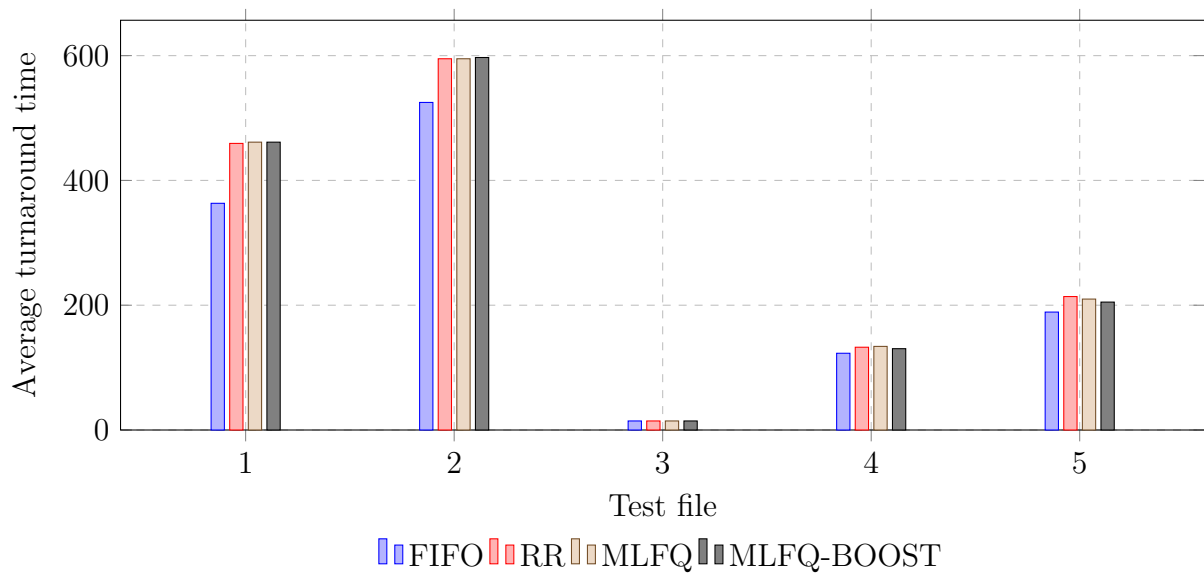


Figure 1: Average turnaround time for 1 CPU.

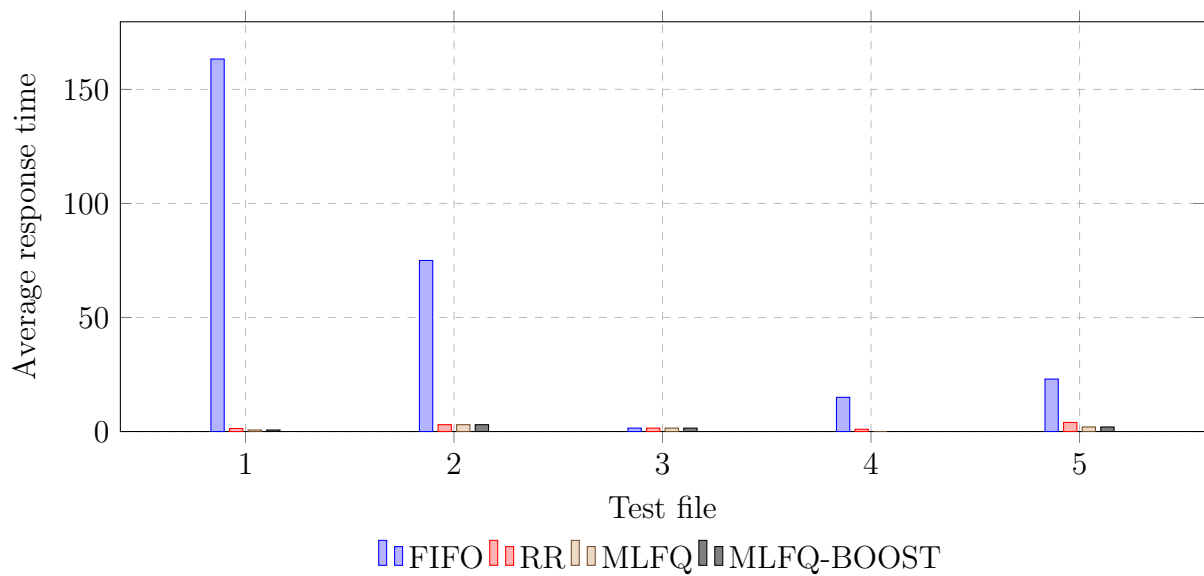


Figure 2: Average response time for 1 CPU.

4.2 Two CPUs

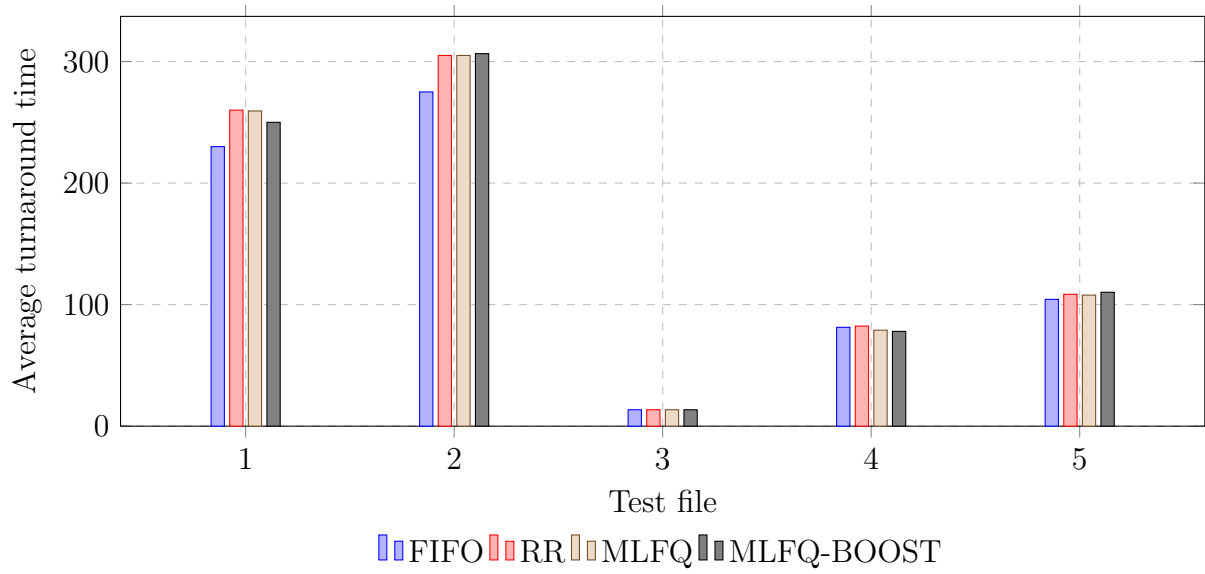


Figure 3: Average turnaround time for 2 CPUs.

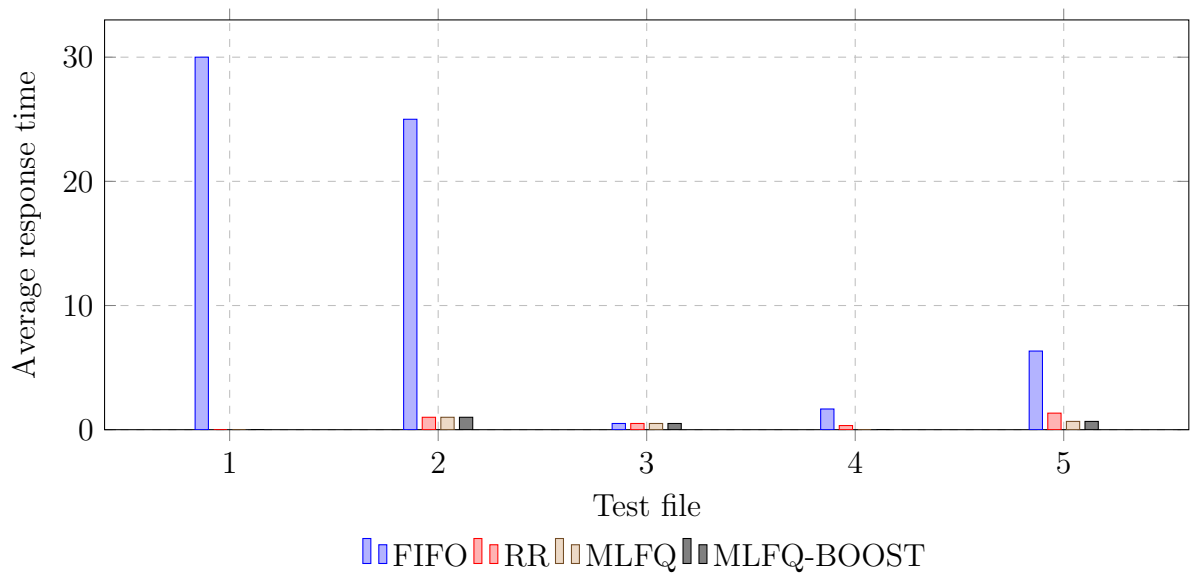


Figure 4: Average response time for 2 CPUs.

4.3 Effect of Round Robin Quantum

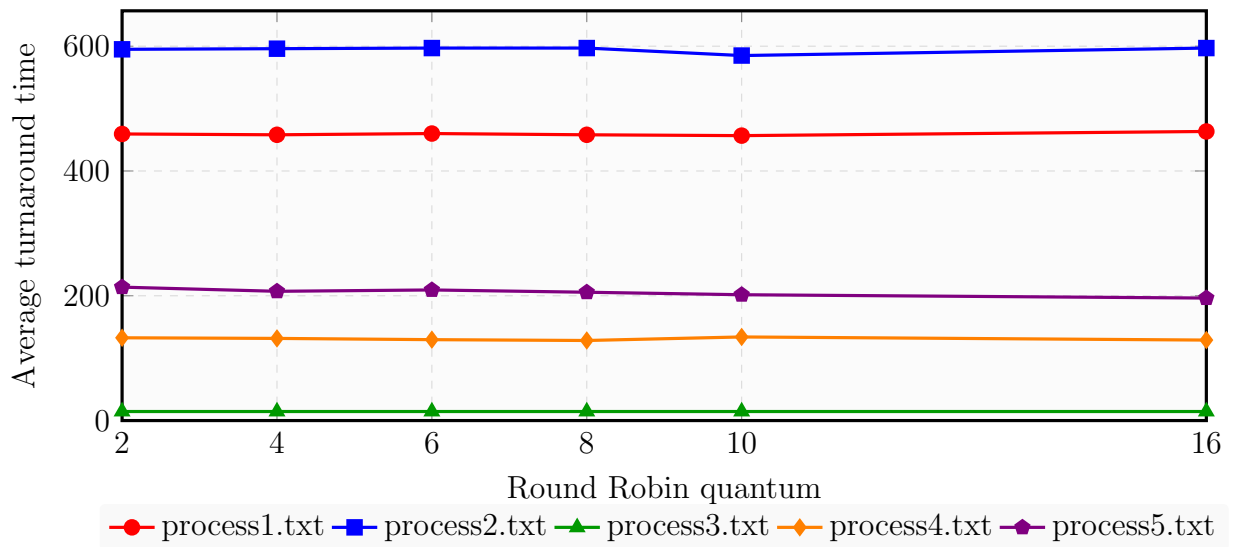


Figure 5: Effect of Round Robin quantum on average turnaround time for 1 CPU.

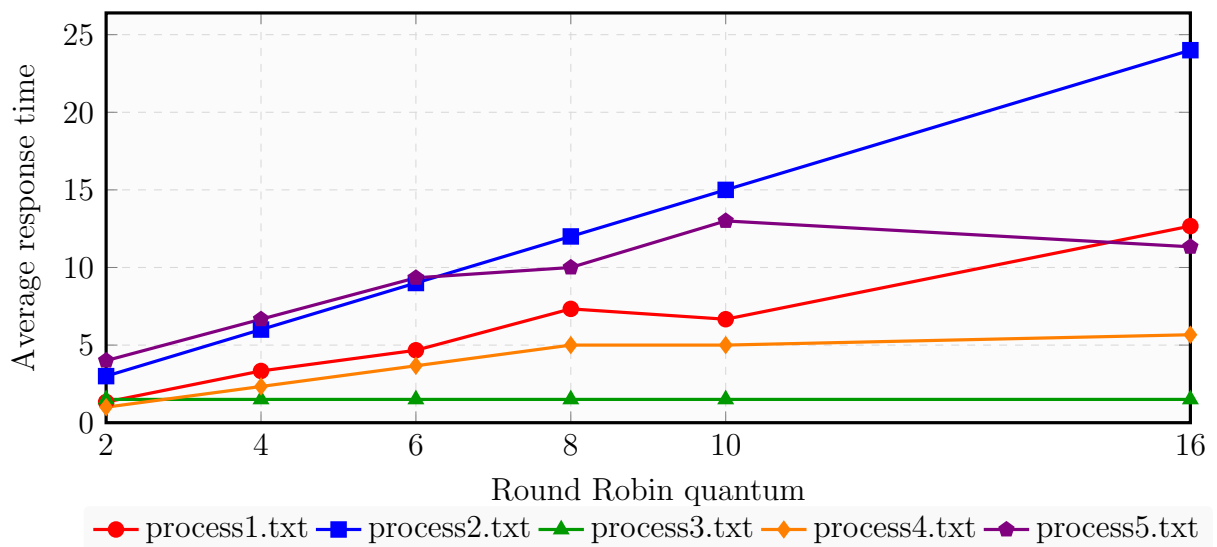


Figure 6: Effect of Round Robin quantum on average response time for 1 CPU.

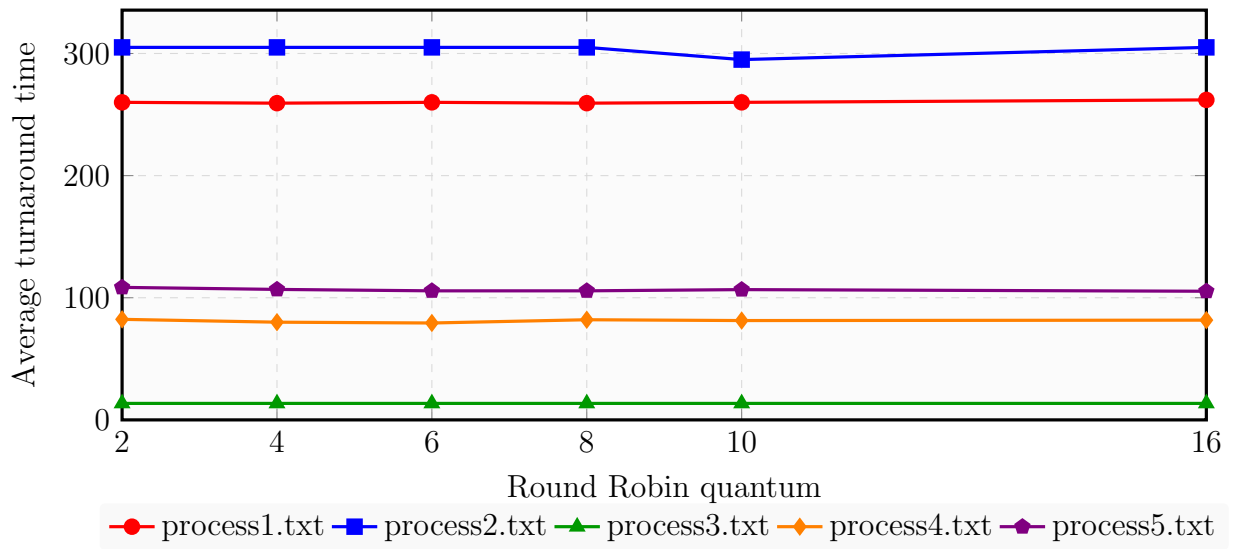


Figure 7: Effect of Round Robin quantum on average turnaround time for 2 CPUs.

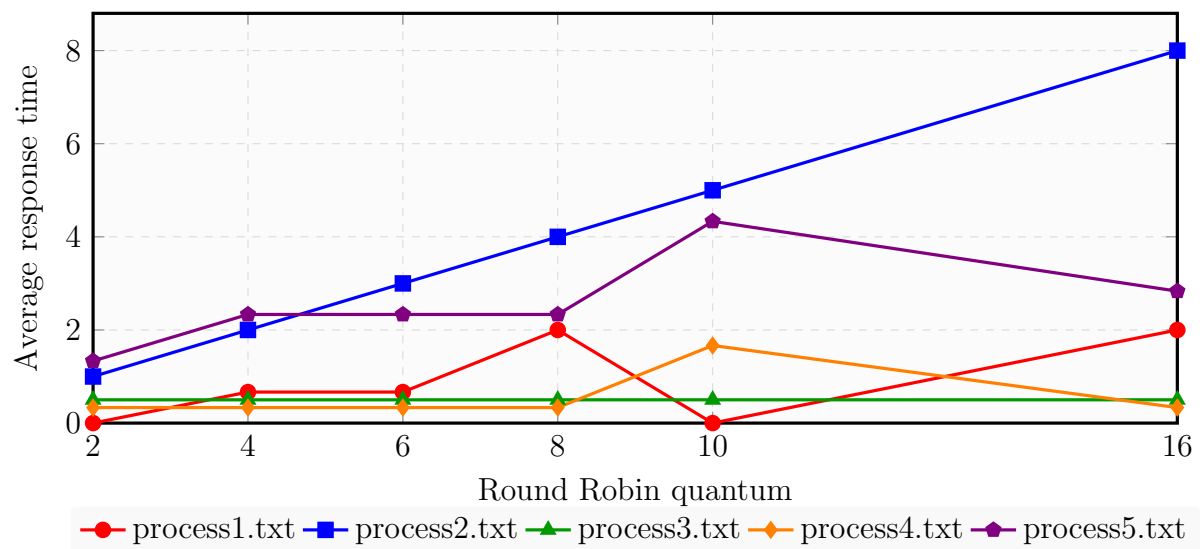


Figure 8: Effect of Round Robin quantum on average response time for 2 CPUs.

5 Inference

FIFO vs. RR vs. MLFQ (single and dual CPU)

- **Response time:** FIFO is the clear outlier, with average response times often two orders of magnitude larger than RR or MLFQ on workloads containing long CPU-bound jobs. This is a direct consequence of FIFO being non-preemptive: a short job that arrives while a long job is running must wait for the entire burst to finish. RR and MLFQ avoid this by preempting at every quantum, so a newly arrived process is scheduled within at most one quantum of its arrival.
- **Turnaround time:** The picture is less one-sided. On workloads dominated by a few long CPU-bound processes, FIFO achieves *lower* turnaround than RR and MLFQ, while on workloads with many short interacting processes the three policies are nearly indistinguishable. This is not a contradiction: RR’s low response time is purchased by interrupting long bursts, which increases those jobs’ completion times. FIFO’s better turnaround on long-job workloads reflects that it lets each job run to completion without paying for repeated requeuing.
- **Effect of CPU count:** Doubling the CPU count roughly halves turnaround time and sharply reduces FIFO’s response time across all policies. This is the expected throughput effect and does not change the relative ordering of policies, so the qualitative conclusions above are robust to the number of CPUs.
- **MLFQ vs. RR:** MLFQ and MLFQ-BOOST are essentially tied with RR on both metrics. With a three-level MLFQ using the same quantum at every level, a process is demoted only after exhausting its quantum—which for short jobs never happens. The priority mechanism therefore has little opportunity to differentiate behaviour, and MLFQ effectively degenerates to RR for these workloads.
- **MLFQ-BOOST vs. MLFQ:** The two are near-identical on every workload. Periodic priority boosting only matters when processes actually get demoted and risk starvation; on workloads where few processes reach the lower queues, boosting has nothing to correct.

Effect of Round Robin quantum

- **Response time grows with quantum.** On most workloads, average response time increases roughly in proportion to the quantum, across both CPU counts. This is expected: a newly arrived process must wait for the processes ahead of it in the ready queue, and each of those runs for up to q time units before being preempted, so longer quanta mean longer waits before a process’s first turn.
- **Turnaround time shows no clear trend with quantum.** Across the workloads tested, average turnaround time varies by only a few percent over the full quantum range, with no consistent direction. The variation is small enough relative to the measurement spread that the data cannot resolve whether the quantum has a systematic effect on turnaround time.
- **Small quanta are preferable when responsiveness matters.** Since turnaround is essentially flat in q while response time grows with q , a small quantum is strictly

better for interactive workloads at negligible cost to completion time. The trade-off is not visible in this simulator because context-switch overhead is not modelled; in a real system q cannot be reduced arbitrarily without paying for switching.

Caveats

- The conclusions above are drawn from five workloads with very different characteristics: some CPU-bound with a few long jobs, some IO-bound with many short interacting bursts, and one too small and uniform to discriminate between policies at all. The last should not be treated as evidence in either direction.
- The simulator does not model context-switch cost, so the response-time advantage of small quanta is overstated relative to a real system.
- MLFQ-BOOST differs from MLFQ only through periodic priority boosting; on these workloads the two are nearly identical, so no strong claim about the benefit of boosting can be made from this data alone. A workload designed to starve a process in a low queue would be needed to exercise the mechanism.